

3.1 运输层提供的服务

运输层为不同主机进程间提供端到端的**逻辑通信**功能(无需考虑具体物理链路)。

运输层协议运行在端系统，不在路由器中。发送方运输层将应用进程的报文划分成若干报文段，封装后传给网络层；接收方运输层将网络层上传的报文段，去掉运输层首部后，重新装配为报文，传向应用层。双方**网络层**负责主机到主机的网络传输，**提供主机之间的逻辑通信**。

运输层协议有TCP和UDP，提供不同的服务质量，所能提供的服务受到网络层提供服务的限制，如不保证时延和带宽。同样，网络层服务是不可靠的(存在分组丢失、失序和重复)，运输层协议通过检错重传、排序、查重等措施能为应用程序提供可靠的传输服务。

网络层的主要协议是IP(网际协议)，TCP和UDP都基于IP。IP提供不可靠的传输服务，尽最大的努力在通信的主机之间交付数据报，但不保证按序交付和数据的完整性。IP使用IP地址，即设备接口的网络层地址，每台主机至少有一个，路由器有多个。

3.2 多路复用与分解

主机上可能同时运行**多个应用进程**，当运输层从网络层接收数据后，**通过套接字**定向到相应的进程，主机上的**套接字可以有多个**，对应不同的进程。

发送主机的多路复用：运输层从不同套接字收集数据，并为**每个数据块封装上运输层首部，生成报文段**，交给网络层。

接收主机的多路分解：运输层从网络层收到报文段，取出运输层首部并识别出套接字，并将剩余数据传给该套接字。

无连接的多路复用与分解：UDP套接字由**二元组(IP地址，端口号)**组成，具有不同源套接字、相同目的套接字的不同UDP报文段，可传送到相同的目的进程。创建UDP套接字可由操作系统分配临时端口号或指定端口号，客户机进程通常使用临时端口号，服务器进程通常使用周知端口号。

面向连接的多路复用与分解：TCP套接字由**四元组(源IP地址，源端口号，目的IP地址，目的端口号)**组成，服务器支持多个并行的TCP套接字，每个新的连接都由服务器创建的新进程负责。创建套接字时，**服务器进程**先创建欢迎套接字，侦听任何客户机的连接请求；**客户机进程**创建客户套接字并发起连接；当服务器收到连接请求后，**创建一个新的套接字与客户建立连接**，老的欢迎套接字继续侦听其他客户请求；TCP连接完成后，客户机和服务器可互相发送数据。

Web服务器可以产生多个进程**同时与多个客户机连接**，每个进程都使用其对应的TCP套接字，收发HTTP报文，高性能Web服务器可能只使用一个进程，但为每个新连接创建一个**新线程(一个轻量级的子进程)**，**建立新的TCP套接字(此时服务器进程的端口号不变)**。

3.3 无连接运输：UDP

UDP提供“尽力而为”的服务，可能丢包或失序。无连接运输通信双方无握手过程，每个UDP段相互独立(而多个TCP段通常属于同一个文件)。

UDP的检验和(checksum)：UDP提供有限的可靠性，检查端到端传输的**比特差错**(注意：会同时检查网络层首部部分字段，如IP地址等)，只检错，不纠错不报错。

发送方将要检查数据按16位分块，检验和字段初始为16个0；将所有块求和(最高位进位加到最低位)；对和求反，得到检验和，放入检验和字段再发送。接收方对收到的相关数据(包括检验和)分块、求和，如果结果全"1"，则数据无错，否则数据出错。



图3-1：UDP的检验和

使用UDP协议的原因在于，不创建连接可减少时延，段首部简单导致传输、处理开销小，无拥塞控制可以尽可能快地传输。UDP主要应用于能容忍数据丢失，小块数据的快速传送，如流式多媒体、DNS、SNMP(简单网络管理协议)、TFTP(简单文件传输协议)、RIP(路由信息协议)。

也有基于UDP的可靠传输，需在应用层采取额外的可靠性措施，实现特定的差错恢复。

3.4 可靠数据传输的原理

可靠数据传输指低层信道不太可靠，本层采取一些措施，为上一层数据提供**可靠传输服务**，即数据不出错，不丢失，按序传送。

可靠传输服务通过分别设计可靠数据传输协议的**发送方和接收方实体**来实现。

发送方：上一层通过**rdt_send()**函数调用本层可靠数据传输协议；本层协议实体通过**udt_send()**使用下层不可靠信道发送分组。

接收方：本层协议实体通过**rdt_rcv()**从下层收到数据，并实现数据可靠性处理；本层通过**deliver_data()**函数将可靠数据交付给上一层。

下为针对不同信道的可靠数据传输协议。

1.完全可靠信道：

rdt 1.0 实现发与收 2.具有比特差错的信道：

rdt 2.0 增加检错：采用确认和重传(使用校验和)

rdt 2.1 增加确认出错的处理：如果确认出错，发送方重传数据分组，接收方检测冗余数据分组(数据分组使用序号)

rdt 2.2 用ACK实现NAK功能(ACK使用序号)

3.具有比特差错和丢包的信道：

rdt 3.0 增加对丢包的处理(发送方使用定时器)

使用有限状态自动机(FSM/FSA,Finite State Machine/Automaton)，可完整地描述协议发送方和接收方实体的工作逻辑。

有限状态机是离散事件系统建模与分析的图形化数学工具，组成如下。

圆圈：表示系统的状态，虚线箭头指向系统的初始状态

带箭头的弧：表示系统的状态变迁

弧的横线上方：表示引起状态变迁的事件

弧的横线下方：表示变迁时系统要采取的动作

Λ 符号：表示零事件或零动作

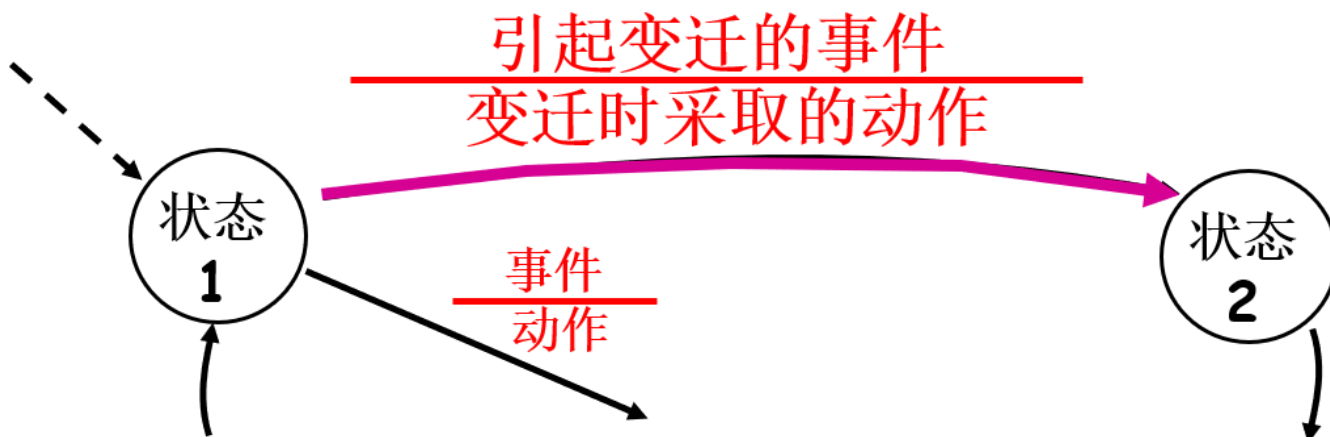


图3-2：FSM

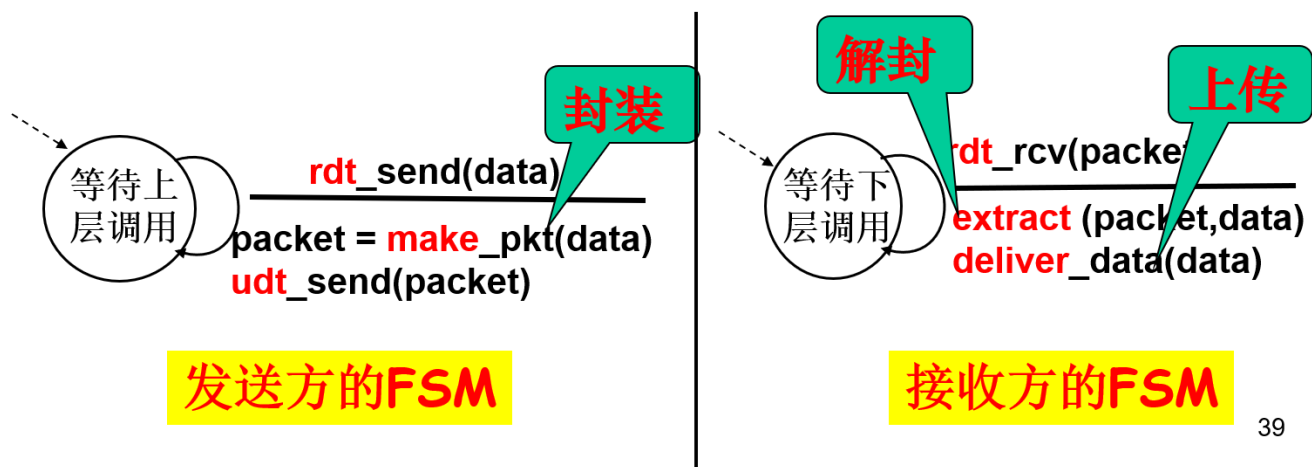
3.4.1 构造可靠数据传输协议

1、可靠信道上的可靠数据传输：rdt 1.0

假设**信道完全可靠**，数据传送不出错不丢包，**收发双方速率匹配**，所以接收方**不用检错和返回确认**。

发送方：将数据发向下一层信道，从上一层接收数据rdt_send(data)；封装成本层分组make_pkt(data)；将分组使用下一层的传输服务udt_send(packet)。

接收方：从下层信道读取数据，从下层接收一个分组rdt_rcv(packet)；解封取出数据extract(packet,data)进行处理；将剩余数据传给上层deliver_data(data)。



39

图3-3: rdt 1.0的FSM

2、具有比特差错信道的可靠数据传输: rdt 2.0

假设下一层信道不完全可靠, 可能产生比特差错, 但不丢失分组。

ARQ(Automatic Repeat reQuest 自动重传请求)协议: 接收方肯定确认, 告诉发送方, **分组已被正确接收**; 接收方否定确认, 告诉发送方, **分组出错, 需要发送方重传**。

基于**检测重传机制**的可靠数据传输协议称为**自动重传请求ARQ协议**。

ARQ协议的原理

检测: 发送方让分组携带特殊字段用于接收方检错。如首部增加一个**检验和(checksum)**字段。

确认: 接收方针对每个分组返回**肯定确认(ACK)**或**否定确认(NAK)**分组。理论上这种分组只需一个比特长, 如用1表示ACK, 0表示NAK。

重传: 发送方收到NAK后, 重传之前分组。

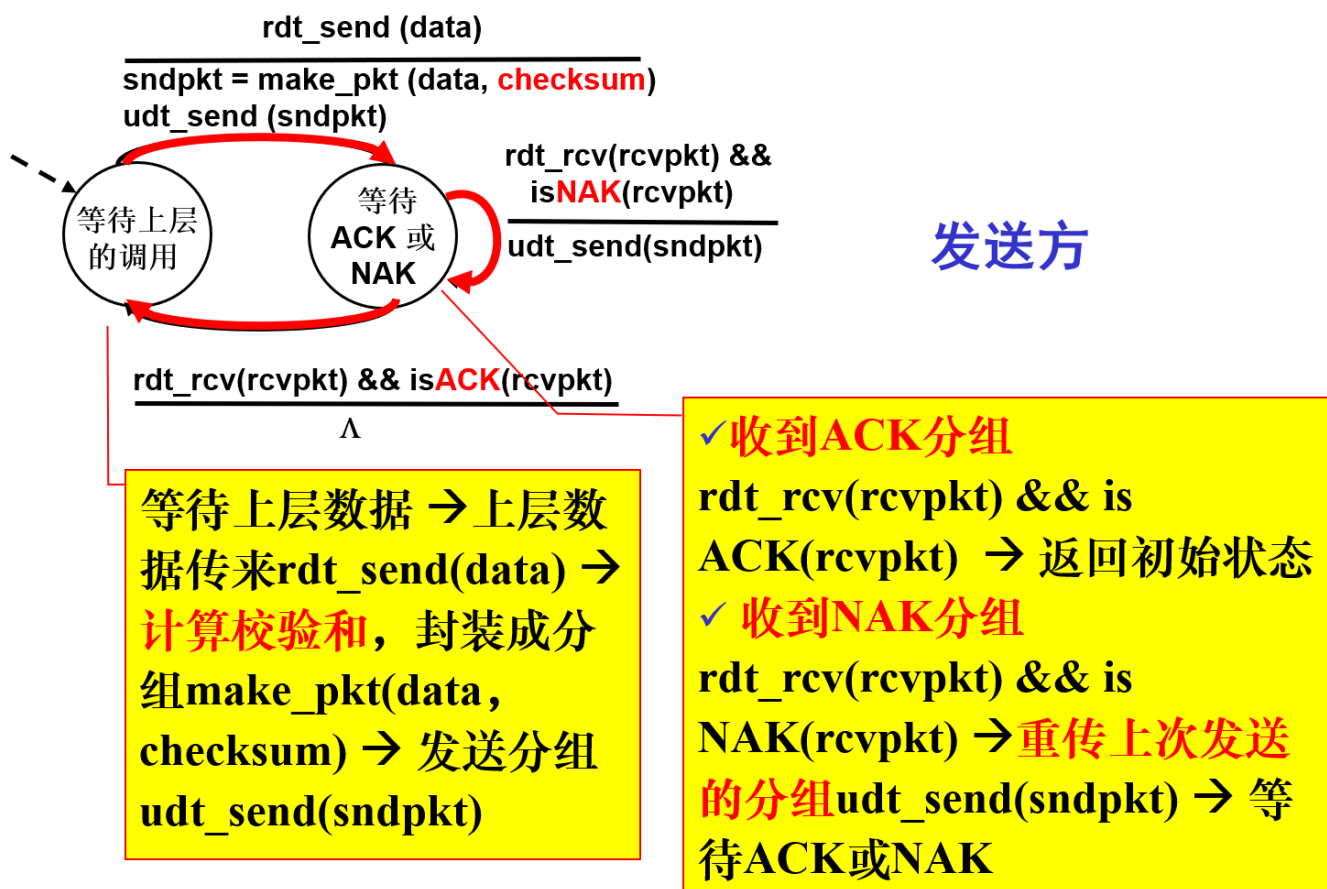
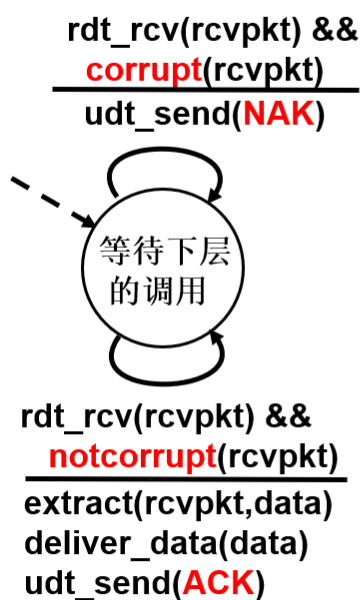


图3-4: rdt 2.0发送方FSM

发送方在**收到ACK分组之前，不会发送任何新分组**，直到收到ACK分组为止。因此rdt 2.0属于**停等(stop-and-wait)**协议，即发送方发完一个分组后停下来，等待对方应答。

分组有错



分组无错

从底层接收一个分组，**检查校验和**

- ✓ **分组出错** rdt_rcv(rcvpkt) && **corrupt**(rcvpkt) → 返回 NAK 分组 udt_send(NAK)
- ✓ **分组无错** rdt_rcv(rcvpkt) && **notcorrupt**(rcvpkt) → 从分组中**取出数据** extract(rcvpkt, data) → 将数据传给上层 deliver_data(data) → 返回 ACK 分组 udt_send(ACK)

图3-5: rdt 2.0接收方FSM

rdt 2.0协议解决了数据出错的问题，但没有考虑ACK确认或NAK确认出错的情况。

对确认出错的处理有两种方法。若**ACK和NAK含加强的检验和**，使发送方不仅可以确认检错，还可纠错。如海明纠错码可纠正一位差错。若**ACK和NAK含普通的检验和**，当发送方收到出错的ACK或NAK时，简单地重发之前的数据分组，是常用方式。但**新问题是**，接收方可能收到冗余数据分组。

对冗余数据分组的处理，可给数据分组首部**添加一个序号字段**。发送方使用序号给数据分组编号，每发送一个新的数据分组，序号加1。接收方检查序号，区别重复数据分组。如重复，丢弃后到的重复分组，并返回ACK。(接收方对正确收到的重复分组也会返回ACK)

3、增加确认出错处理的可靠数据传输：rdt 2.1

停等协议一次只传一个分组，直到收到正确确认才传下一个。此时序号只需一位比特，就可以区分正在传送的数据分组与其它分组。**rdt 2.1中ACK和NAK确认分组无序号**，因为停等协议中，发送方知道，它们是对最近发送数据分组的确认。**FSM状态数翻倍**，用于区分数据分组0或数据分组1的不同状态。

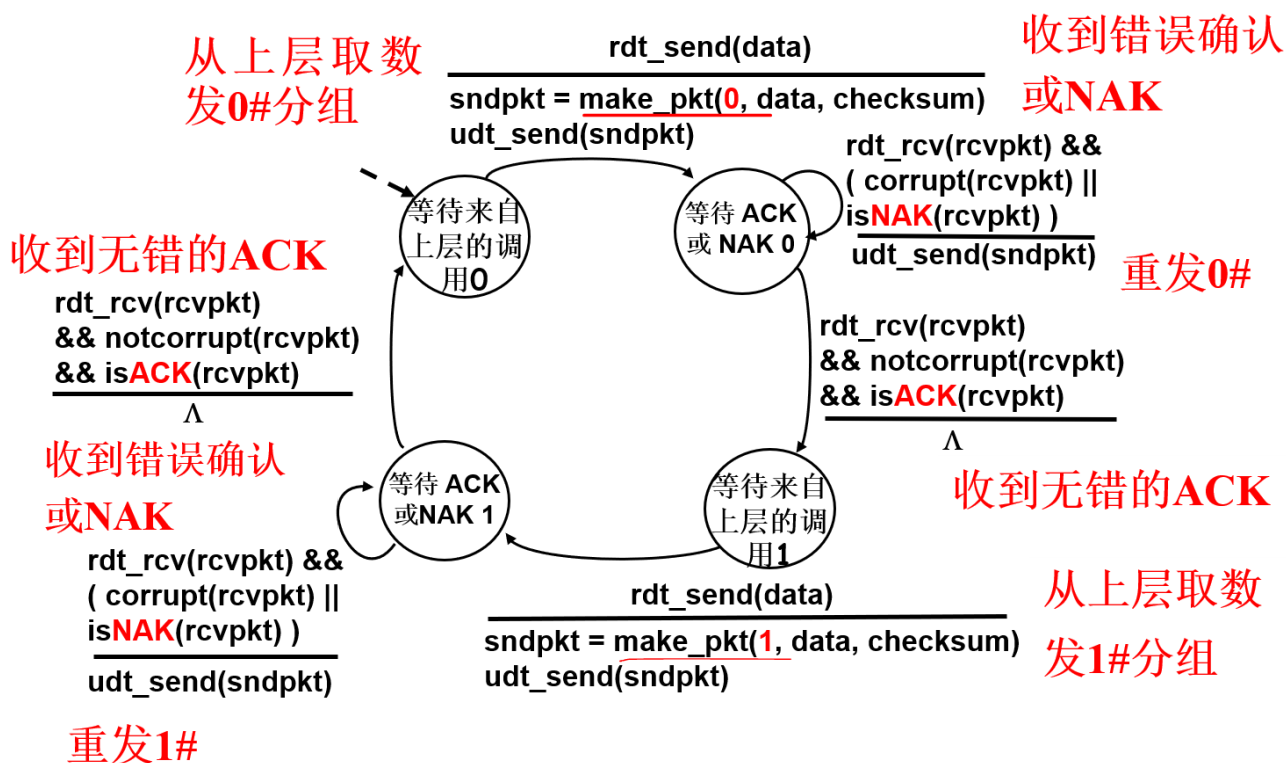


图3-6: rdt 2.1发送方FSM

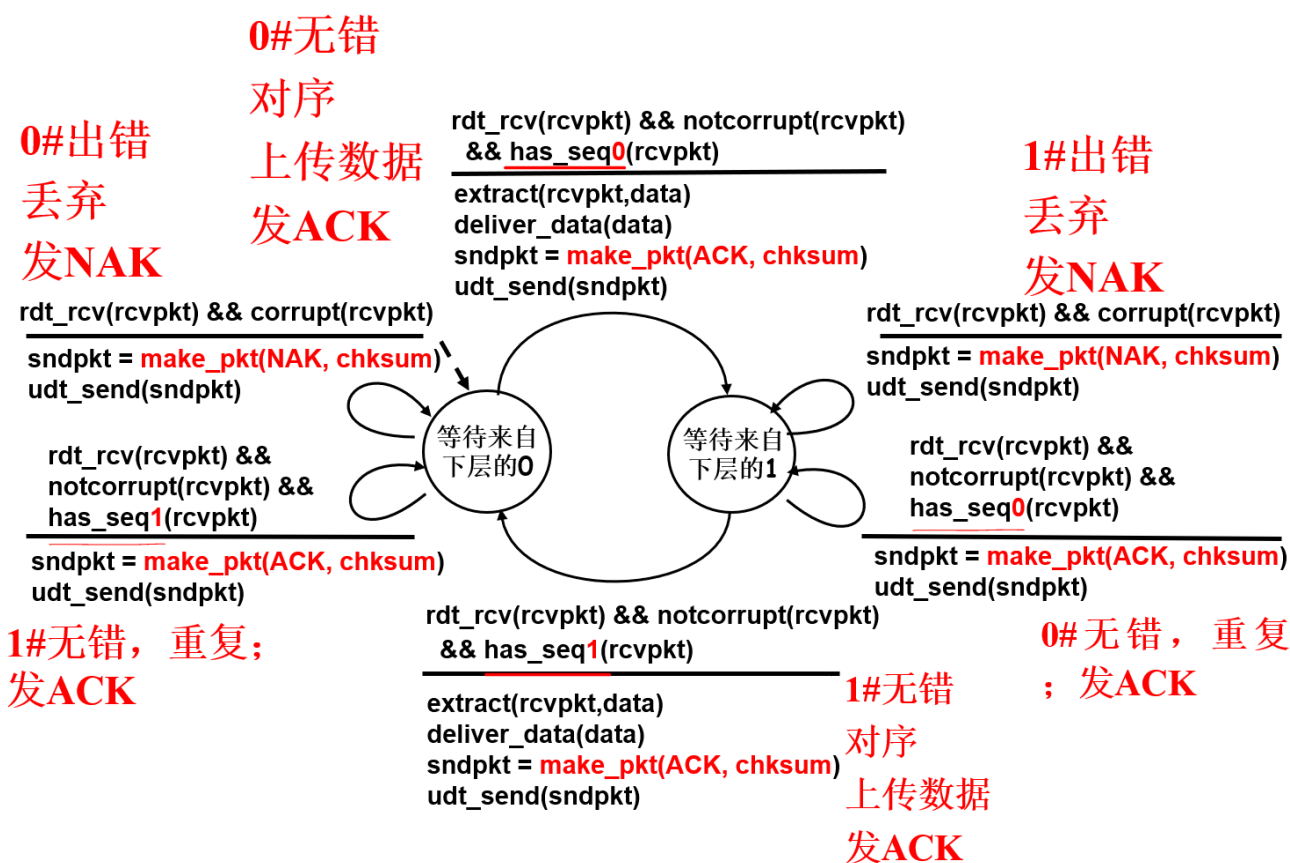


图3-7: rdt 2.1接收方FSM

通过在ACK确认分组中增加序号字段，实现无NAK的可靠数据传输。

接收方接收到出错数据分组时，丢弃，不发送NAK，而发送前一个数据分组的ACK。(和收到重复数据分组的处理一样)

发送方收到前一个数据分组的ACK，则重发当前数据分组。

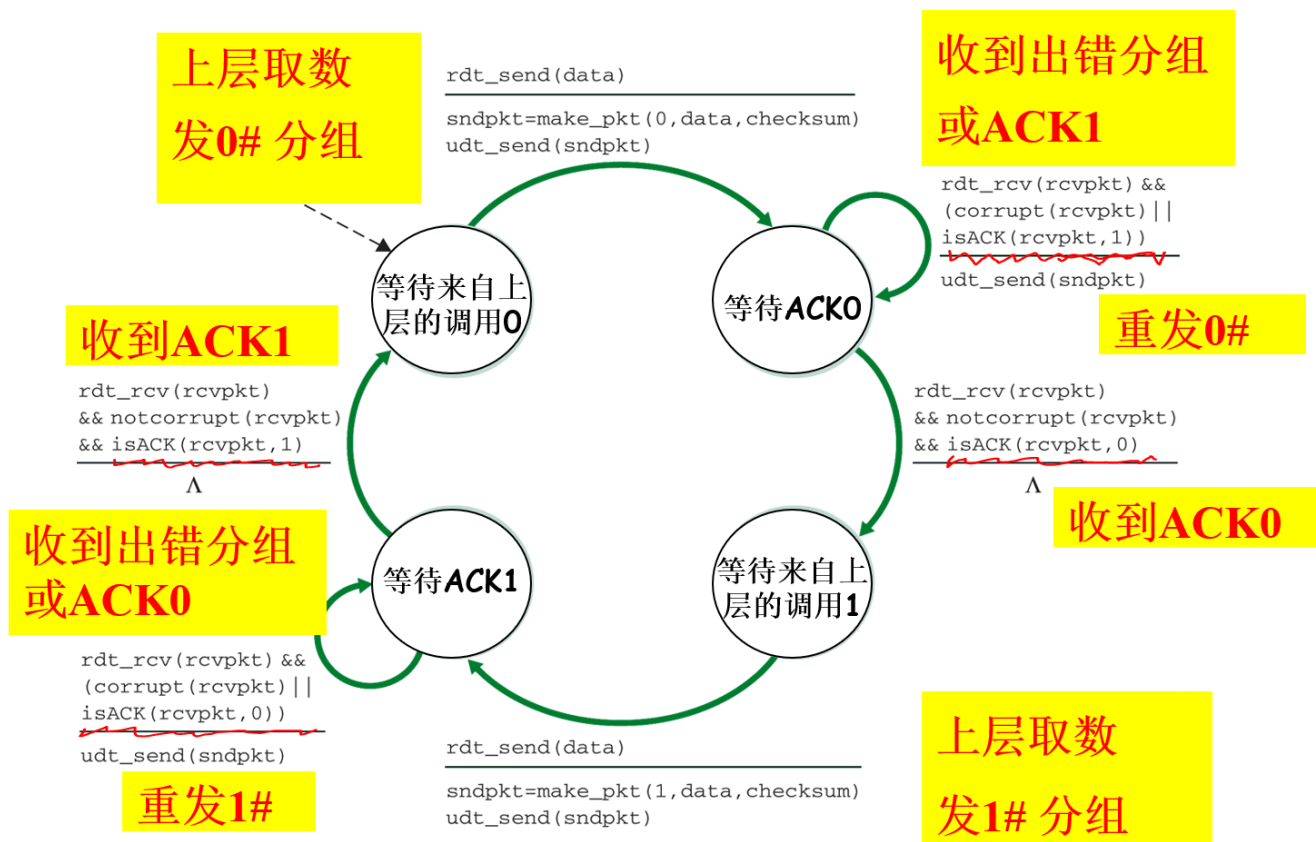


Figure 3.13 ♦ rdt2.2 sender

图3-8: rdt 2.2发送方FSM

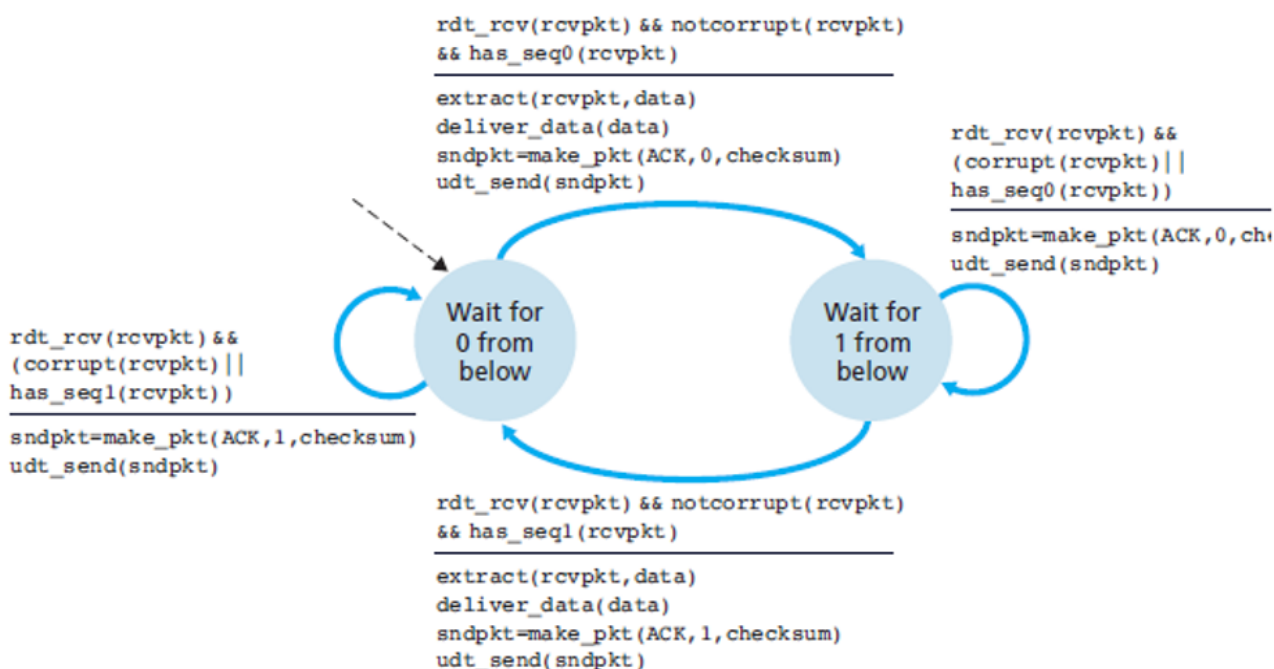


图3-9: rdt 2.2接收方FSM

5、具有比特差错且丢包信道上的可靠数据传输：rdt 3.0

考虑信道会出现分组差错，并且可能出现分组丢失，如发送方的**数据分组丢失**，接收方的**ACK分组丢失**。rdt 3.0通过**在发送方增加定时器处理丢包**。

超时重发：发送方有一个定时器，发送一个数据分组，就启动定时器，**如在超时未收到其ACK，则重传数据分组**。

超时时间选择：通常为往返平均时间的两倍。

新问题：一个数据分组若具有大的时延，导致发送方超时重传，使得接收方会收到**冗余数据分组**。

解决：接收方通过序号判断重复分组(rdt 2.2接收方已实现，因此rdt 3.0接收方的FSM与2.2相同)。

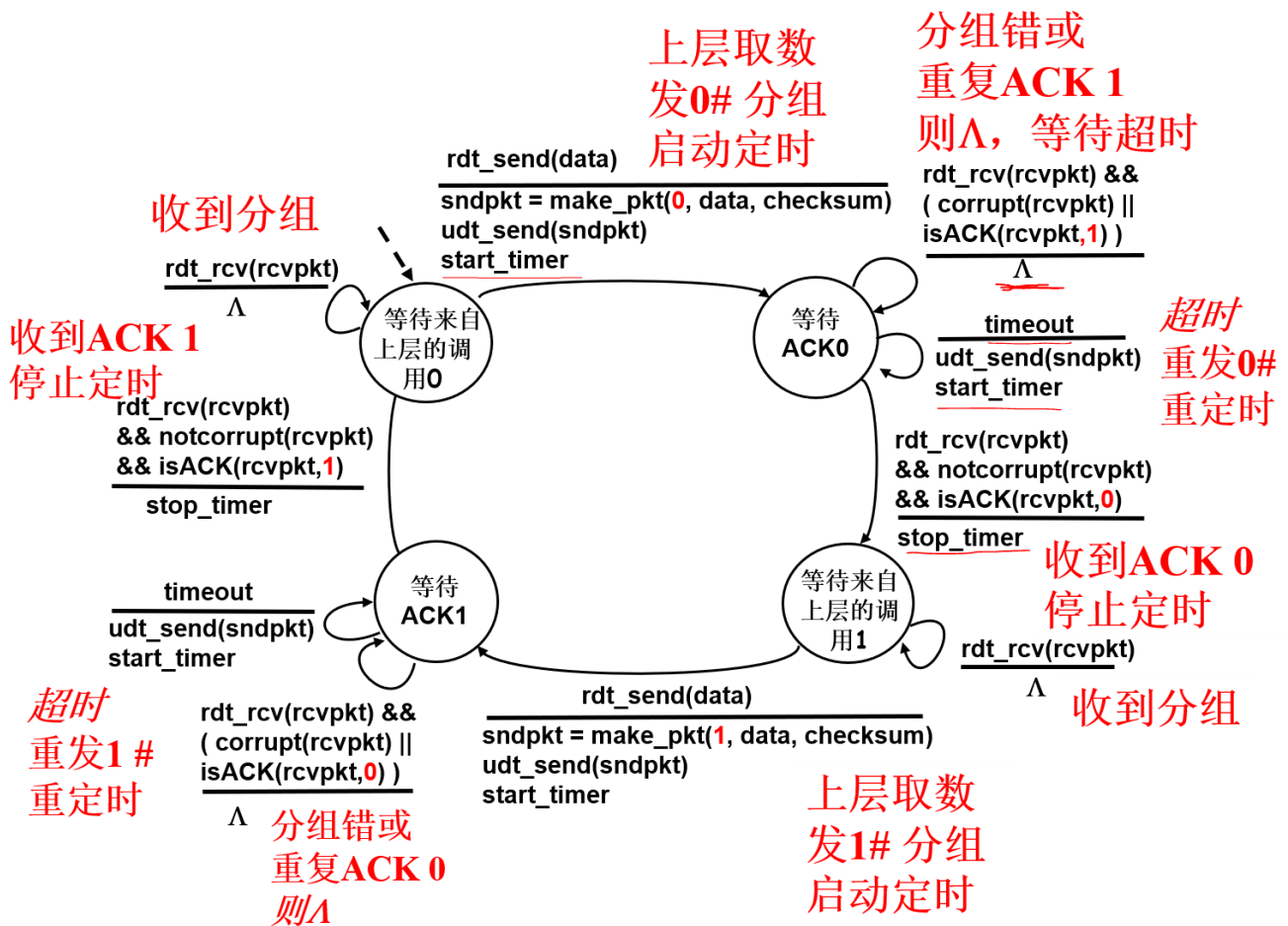


图3-10: rdt 3.0发送方FSM

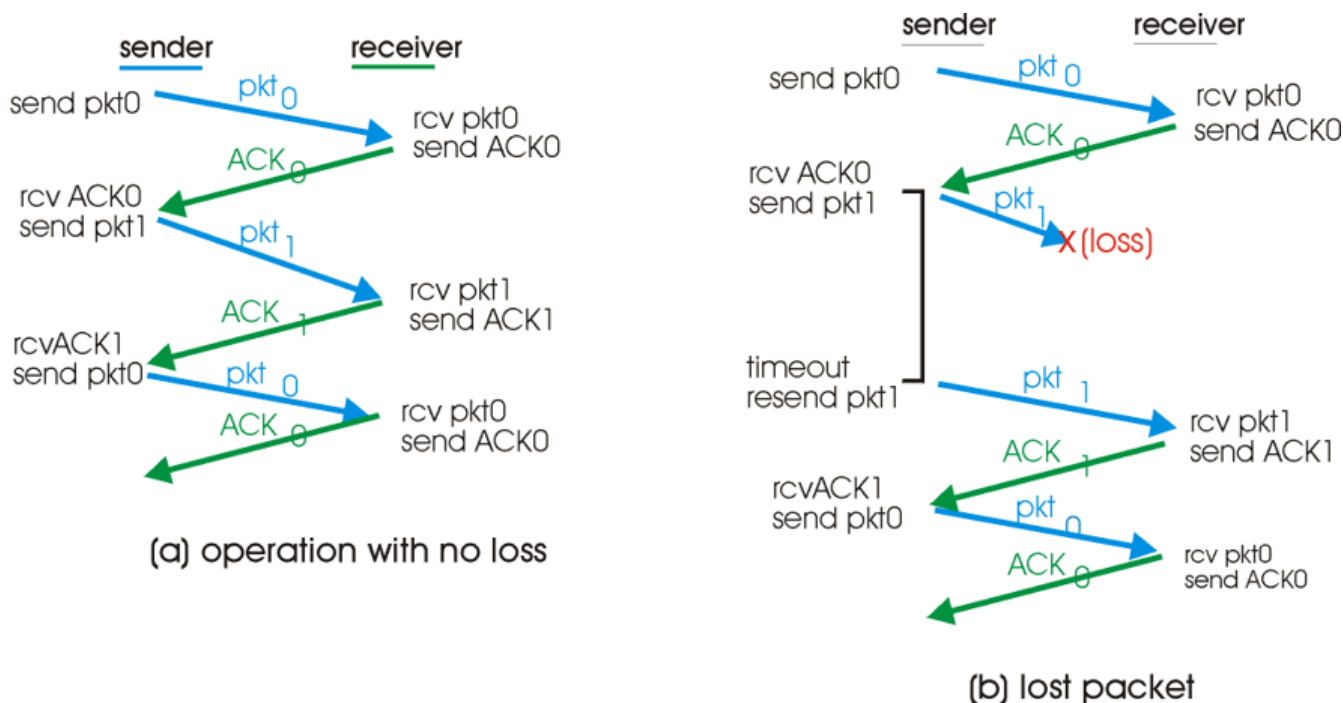


图3-11：rdt 3.0运行情况一

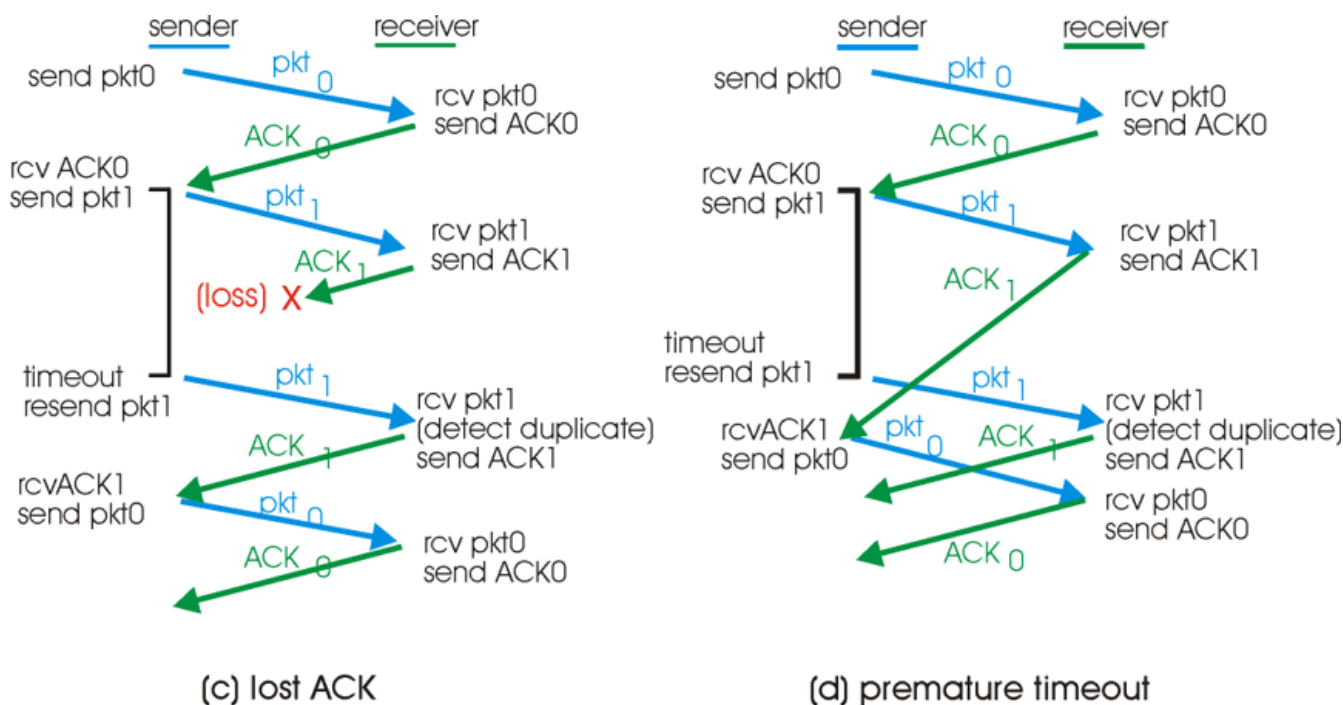


图3-12：rdt 3.0运行情况二

保证可靠数据传输的措施：损坏：检验确认重传。冗余或乱序：序号。丢失：定时器(超时重传)。

rdt 3.0是一个停等协议，属于流量控制，能够正常工作，但效率不高。

发送方的信道利用率 $U_{sender} = \frac{\text{发送方将分组发送到信道的时间, 传输时延 } L/R}{\text{通信使用信道的时间, 发送方从发送一个数据分组开始, 到收到对方 } ACK \text{ 分组所需的时间}} \circ$

对停等协议进行改进，允许发送方连续发送多个分组而无需等待确认，称为**流水线技术**。

3.4.2 流水线可靠数据传输协议

令 N 为连续发送分组个数，则其信道利用率为

$$U_{sender} = \frac{L/R}{L/R + RTT} N$$

使用流水线技术有下列要求

需要增加序号范围：区分连续发送的各分组。

需要缓存：**发送方**缓存已发送但未收到确认的分组(可能要重发)；**接收方**缓存已正确接收但还未向上层提交的分组。
两个缓存大小取决于所用的**流水线差错恢复技术**。

流水线的差错恢复技术如下

回退N步(Go-Back-N GBN协议)：发送方缓存大小为N个分组，接收方缓存大小为1个分组。

选择性重传(Selective Repeat SR协议)：发送方和接收方缓存都为N个分组。

停等法的差错恢复技术(rdt)：发送方和接收方缓存都为1个分组。

3.4.3 回退N步

GBN(Go-Back-N 回退N步协议)：

发送方：可连续发送多个数据分组(始终要保证未确认的分组数不超过N，即发送方缓存大小)，停止等待，并启动计时器。如收到某未确认分组的ACK，继续发送后续可发送分组；如超时，从未收到ACK的第一个分组开始重发。

接收方：最终实现按序向上层提交数据。如收到正确且有序分组，放入缓存，准备向上层提交，并返回此分组的ACK；收到出错或失序分组，每次返回按序正确收到的最后一个分组的ACK。

发送方可连续发送的分组数不能超过发送方缓存的大小N，即发送方已发送未确认的最大分组数，因此，通过设置**发送窗口**(对应发送方缓存)和**接收窗口**(对应接收方缓存)，实现**流量控制**。

1、发送窗口($W_s = N$)

指发送方缓存，存放已发送未确认和当前可以发送的分组，用于控制发送速度。

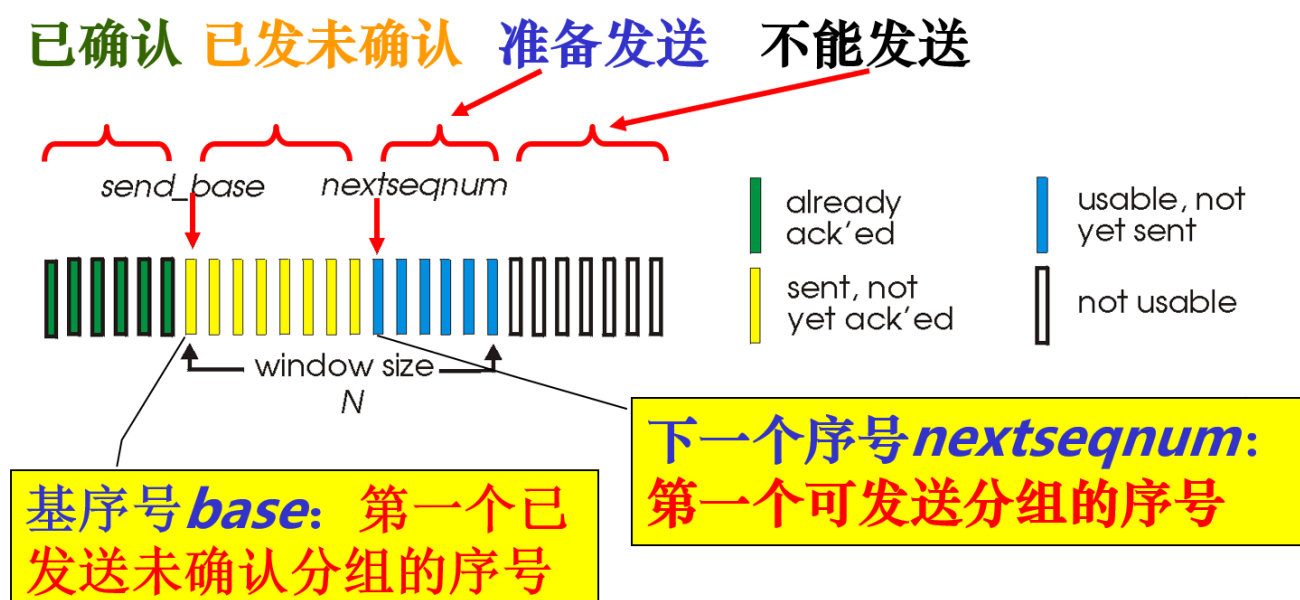


图3-13：发送窗口

发送方的分组：

$[0, base - 1]$: 已发送并收到确认的分组。

$[base, nextseqnum - 1]$: 已发送未确认的分组。

$[nextseqnum, base + N - 1]$: 当前可发送的分组。

$\geq base + N$: 不能发送, 除非收到新的确认, 使得发送窗口向右移动。

发送窗口大小 $W_s = N$, 即序号范围 $[base, base + N - 1]$, 包括**已发送未确认**以及**可立即发送**的分组。

随着已发送的分组陆续被确认, 发送窗口在序号空间中向右滑动, 所以GBN协议又称为**滑动窗口协议**(sliding-window-protol)

窗口已满(窗口中全都是已发送未确认分组): $nextseqnum = base + N$ 。

窗口未满(有可立即发送的分组): $nextseqnum < base + N$ 。

2、分组序号的取值范围

如果序号位数为 k , 其序号范围是 $[0, 2^k - 1]$, GBN发送窗口大小至少要比序号个数少一:

$$W_s = N \leq 2^k - 1$$

停等协议是一个特殊的GBN协议(即已发送未确认的分组数不能超过1), 其序号的位数为 $k = 1$, 序号范围是 $[0, 1]$, 发送窗口大小为 $W_s = N = 1$ 。

3、GBN发送方

上层的调用: 判断**发送窗口是否已满** $nextseqnum = base + N$? 若**已满**, 则无法发送, 以后再试; 若**未满**, 则封装并**发送**, $nextseqnum$ 加1, 如果是窗内第一个分组 $base == nextseqnum$, 启动定时器。

超时: 发送方有一个**定时器**, 用于**第一个已发送未确认分组**, 此分组位于窗口的开始位置, 即基序号位置。超时后, **重发**所有从基序号开始的已发送分组, 重定时。

收到ACK: 表明对应分组及其之前分组已被正确有序接收(即**累计确认**)。并将窗口移动, 使得基序号移动到最小未确认分组处, 此时, 若窗口为空(没有已发送未确认分组), 则停止定时器, 否则重启定时器。

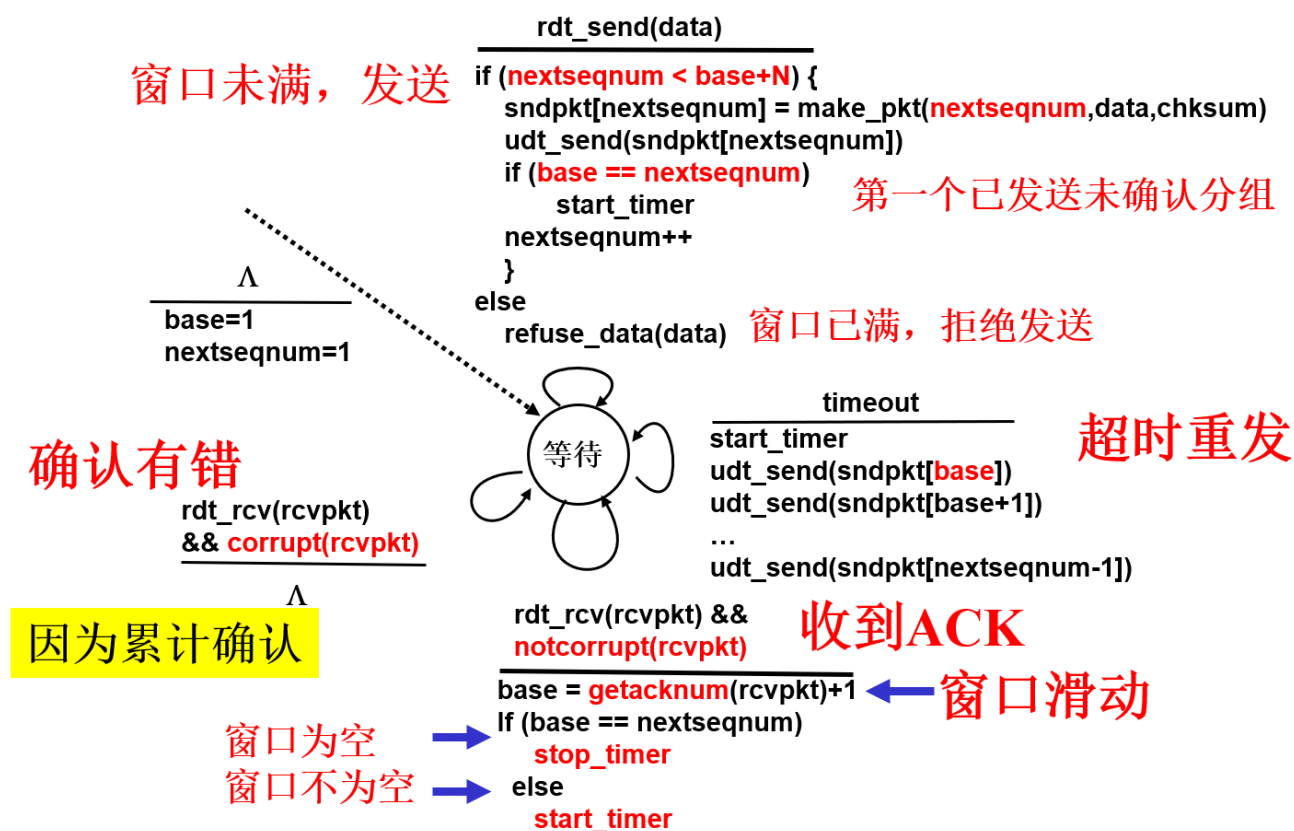


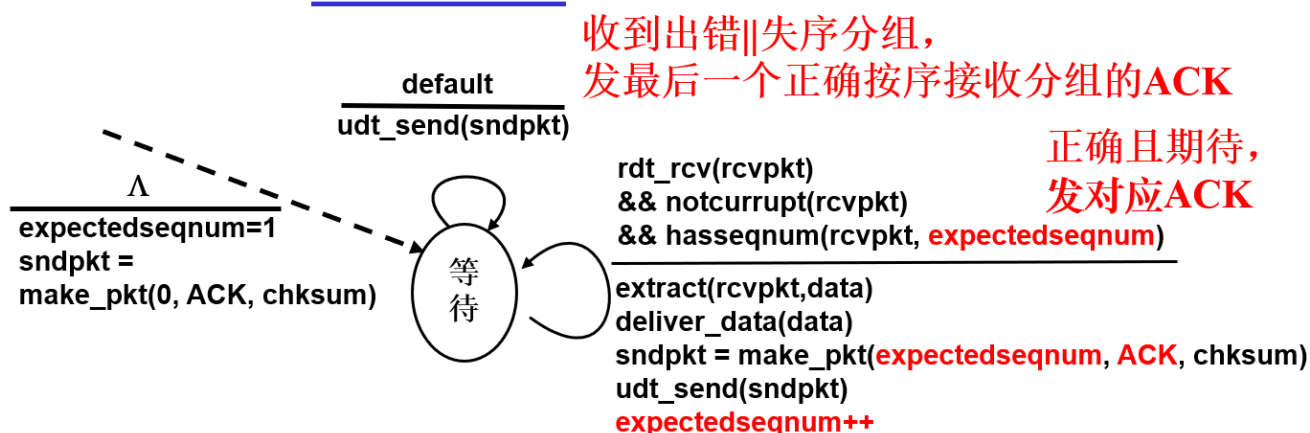
图3-14: GBN发送方FSM

4、GBN接收方

接收方窗口(缓存) $W_R = 1$, 采用**累计确认**, 即返回的ACK对应目前已正确按序接收的最后一个分组。

若收到正确且序号为期待序号的分组, 则将分组放入缓存等待向上提交, 并返回此分组ACK; 若收到出错或失序(序号超前或滞后)的分组, 则返回正确有序收到的最后一个分组的ACK。

GBN接收方的FSM



- ✓ 接收方 **expectedseqnum(期望序号)**: 给出期望按序收到的下一个分组的序号。
- ✓ 接收方返回的ACK: 累计确认。

GBN缺点: 发送方可能发送较多的重传分组

图3-15: GBN接收方的FSM

5、GBN发送窗口的大小

若序号位数 k 位，发送窗口大小 $W_S = N \leq 2^k - 1$ ，而不是 2^k 。如序号位数3位，则序号为0~7。GBN发送窗口最大尺寸是7，而不是8。是为了防止ACK全部丢失的情况下，收方误认为重发分组为新分组。

GBN的可靠性措施：使用检验和、累计确认、序号以及超时重传。

3.4.4 选择性重传(SR)

GBN的不足在于，定时器超时后，发送方重传所有已发送未累计确认的分组。当窗口和带宽时延积都很大时，重传的分组会很多，影响效率。

SR(Selective Repeat)对此改进，采用非累积确认，发送方定时器超时后，只重传已发送未确认(非累积确认)的分组。

1、SR发送方

从上层收到数据：检查发送窗口是否已满，未满则发送，否则拒绝发送。

超时：发送方这对每个分组都有一个定时器，若超时，重发对应分组。而GBN中，超时后重发所有已发送未累计确认分组。

收到ACK: (非累积确认)将对应分组标记为已确认；如果ACK序号等于发送基序号，则发送窗口移动，使得基序号移动到最小未确认分组处，并重启定时器。如新窗口内可发送未发送分组，应立即发送。

2、SR接收方

接收窗口 $W_R > 1$ 个分组，可存放序号在接收窗口内的乱序且无错分组。直到所有序号更小的分组被收到为止，再一起交付给上层。所以从上层看，是可靠的传输(正确且有序)。

若正确收到的分组序号在 $[rcv_base, rcv_base + N - 1]$ ，则返回此分组的ACK。若该分组以前没收到过，则缓存起来；若该分组序号等于接收基序号，则将该分组与后续已缓存的连续分组一起交付给上层，接收窗口移动，使基序号移动到最小未缓存分组处。

若正确收到的分组序号在 $[rcv_base - N, rcv_base - 1]$ ，则返回此分组的ACK，避免因ACK丢失，导致发送窗口无法移动。

其他情况(其他序号或受损)，则忽略。

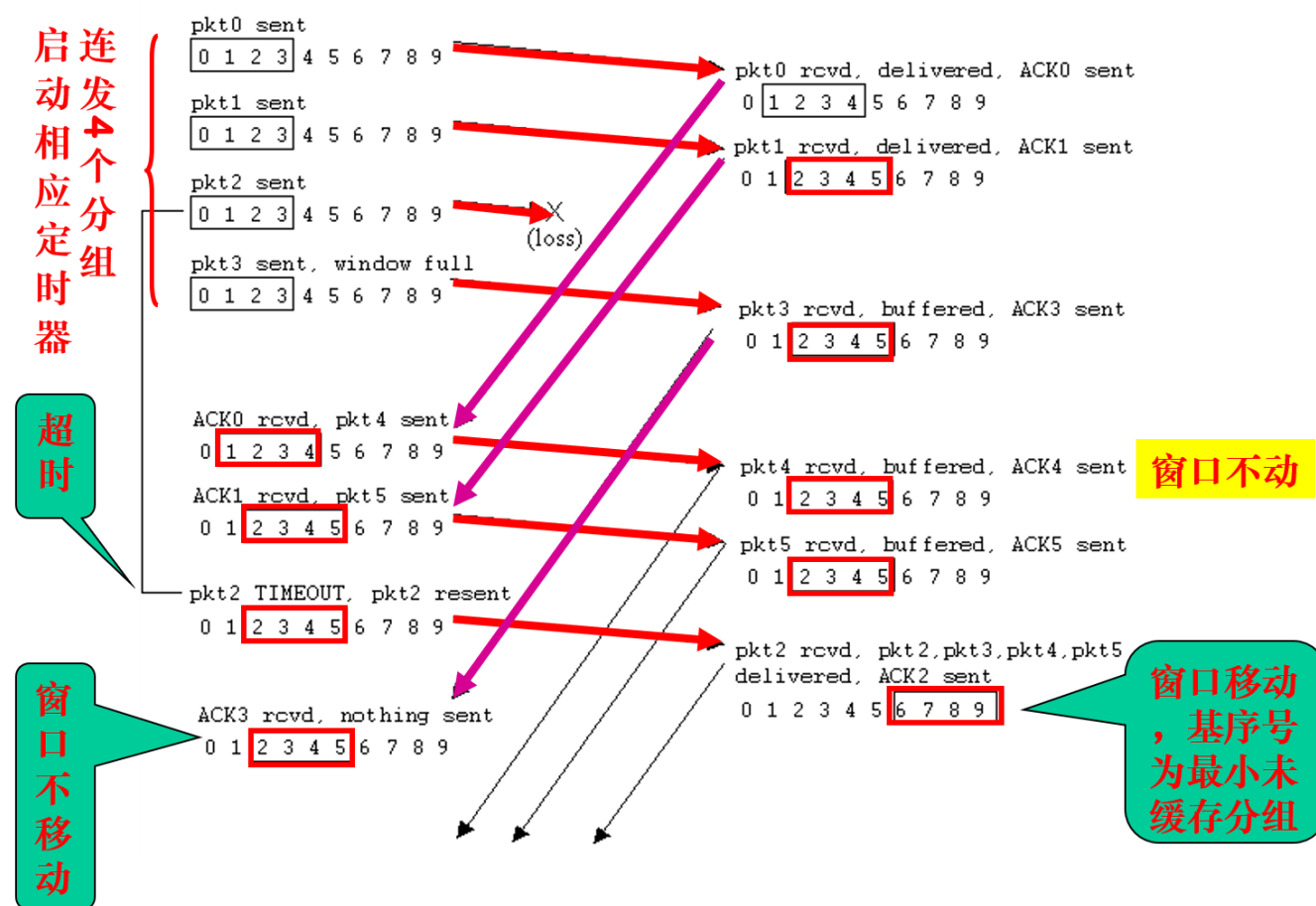


图3-16: SR过程举例

3、SR窗口的大小

若序号位数 k 位，SR发送和接收窗口最大为 2^{k-1} ，不是 $2^k - 1$ 。如序号位数 2 位，则序号为 0~3，SR发送窗口和接收窗口尺寸最大是 2，不是 3。是为了防止 ACK 全部丢失时，收方将重发的分组当成新分组。

4、三种协议比较

	停等协议rdt2.x 3.0	GBN协议	SR协议
发送方	发1个分组，等待确认；重发出错分组	最多连发n个；一个定时器，指向基序号分组；超时，重发窗口中所有已发送未累计确认分组	最多连发n个；每个分组一个定时器；超时，只重发对应分组
接收方	只能按序号 接受；不缓存乱序分组	只能按序号 接受；累计确认；不缓存乱序分组；返回最后一个正确有序接收分组的ACK	可不按序号 接收；非累积确认；缓存窗口内乱序分组；收到当前接收窗口或前一窗口内正确分组，返回对应ACK
窗口大小	$W_S = W_R = 1$	$1 < W_S \leq 2^k - 1; W_R = 1$	$1 < W_S, W_R \leq 2^{k-1}$

5、可靠传输机制

校验和：用于检测分组错误

确认：接收方告诉发送方某个分组或一组分组已被正确地接收。可以是**逐个或累计确认**。

否认：接收方告诉发送方某个分组尚未正确地接收。

序号：**发送的数据分组按顺序编号**。用于检测分组重复或乱序。

定时器：实现发送方**超时重传**，用于检测数据分组或ACK丢失，新问题是超时重传可导致发送方重复传送，接收方通过序号鉴别。

窗口、流水线：发送方、接收方可以**连续发送或接收**序号落在窗口内的分组，而不用等待。

流量控制：控制发送方发送窗口的大小，使得接收方空闲缓存来得及接收。

拥塞控制：控制发方速度(也是通过控制发送窗口实现)，使网络来得及传送。

即使发送和接收窗口都没有序号 x ，但由于网络延时，**接收方可能收到一个以前的序号 x 的分组**，为避免此情况，TCP**规定分组在网络中的最长寿命**大约3分钟。

TCP采用以上所有措施。

3.5 可靠的传输层协议：TCP

3.5.1 TCP连接

TCP协议

面向连接：数据分组传输前，发送方与接收方进行**三次握手(交换控制分组)建立连接**。TCP连接不是电路或虚电路形式，它是基于网络层的IP协议，无固定路径的一种连接(虚连接)。

全双工服务：连接上的**双向**数据流。

点对点：不能多播。

流量控制：控制发送方发送速率，使得接收方来得及接收。

拥塞控制：控制发送方发送速率，使得网络来得及传输。

TCP通信过程

建立TCP连接：**三次握手**，交换控制参数，第三次握手时已经可以传数据了。

进程通信：通过**套接字**。封装成**报文段**传输。通过校验和检测，确认，重传，序号，定时器，流量控制，拥塞控制，在无连接的IP之上实现面向连接(虚连接)的进程间可靠的传输服务。

3.5.2 TCP报文段结构

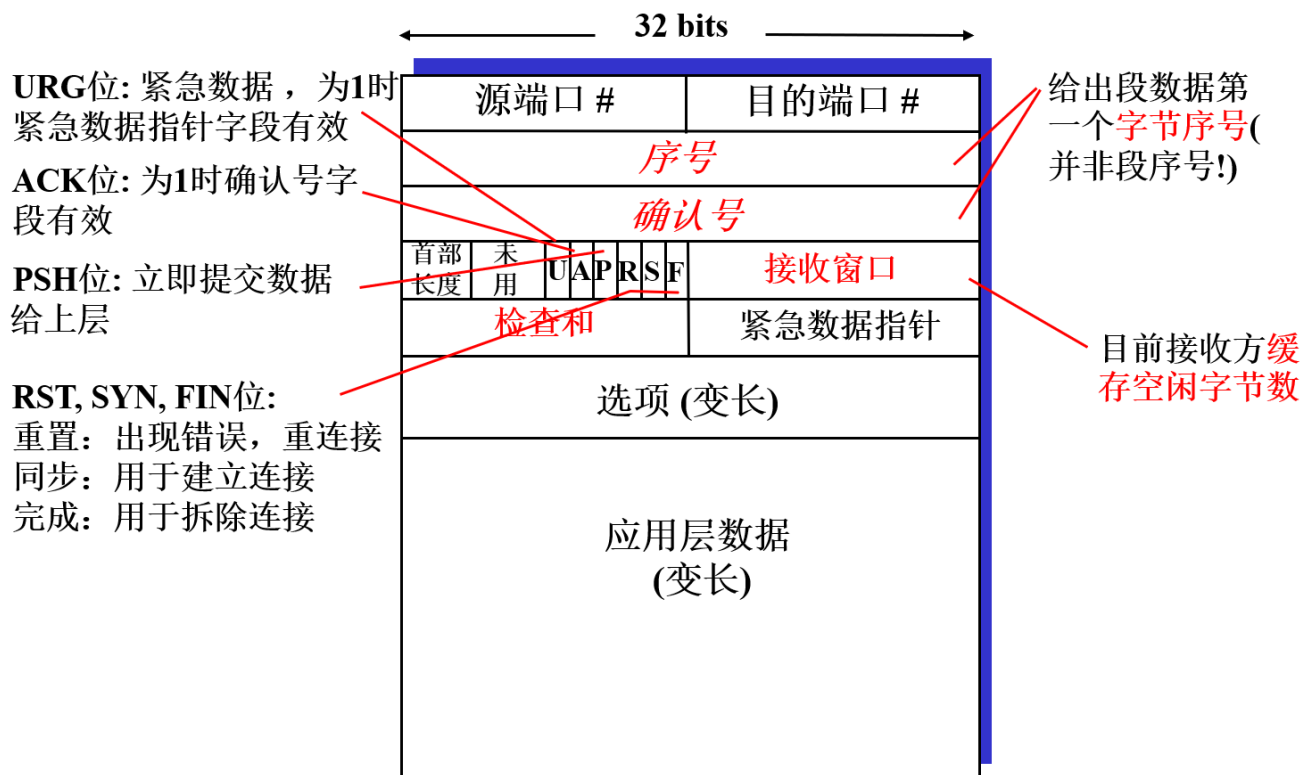


图3-17: TCP报文段结构

TCP把进程数据看成是一个字节流。

TCP序号：报文段中**第1个数据字节**在字节流中的字节序号。

TCP确认号：**期望收到对方下一个字节的序号**；累计确认。

TCP是**全双工的**，两个方向上都有**独立的序号**，且起始序号是随机的。

3.6 拥塞控制

拥塞：太多的**源发送太多太快**的数据，使网络来不及传输。表现为**长时延**(路由器缓冲区排队)或**丢包**(路由器缓冲区溢出)。解决方法有**重传(恶性循环)**或在**拥塞时，控制相应发送方的速率**。

3.6.1 拥塞的原因与代价

代价一：当分组到达路由器速率接近链路容量时，分组**时延**将非常大。

代价二：路由器缓存溢出，会丢包，并引起发送方超时**重传**。

代价三：分组的时延大导致**重传重复数据**。

代价四：当一个分组被丢弃时，每个上游路由器的转发工作成为**无用功**。

所以拥塞控制意义重大，是组网技术中十大重要问题之一。

3.6.2 拥塞控制的方法

根据**网络层是否参与运输层的拥塞控制**，分为端到端拥塞控制(网络层不参与)和网络辅助的拥塞控制。

端到端的拥塞控制：

网络层没有提供支持，端系统必须对网络进行观察(如通过超时机制)来判断网络是否永拥塞。如因特网的TCP拥塞机制。

网络辅助的拥塞控制：

网络层的路由器向发送方提供拥塞反馈信息。如ATM网络的可用比特率ABR(Available Bit Rate)的拥塞控制。路由器的两种反馈信息是用一个比特告知是否拥塞和告知本路由器能在支持的输出速率。

信息反馈的方式有直接反馈，即**路由器发送拥塞分组告知发送方**；经过接收方的反馈，即路由器对去往接收方分组的某个字段进行**标记**，来指示是否拥塞，**接收方收到后，再通知发送方网络是否拥塞**(至少要一个完整的往返时间)。

3.6.3 ATM ABR拥塞控制

ATM异步传输模式(Asynchronous Transfer Mode)：是一种采用网络辅助的拥塞控制的案例。ATM是建立在**电路交换和分组交换**的基础上的一种面向连接的快速分组交换技术，采用**虚电路VC**，从源到目的路径上的每台交换机**维持虚电路VC状态**，可以跟踪各个发送方的行为，并采取源特定拥塞控制动作。

ABR可用比特率(available bit-rate)：是一种**弹性**数据传输服务。网络**轻载时**，ABR服务会充分利用空闲的可用带宽；网络**拥塞时**，ABR服务会将其传输速率**抑制**为预先确定的最小传输速率。

ATM ABR拥塞控制的框架

信元即分组，是ATM术语。

数据信元：从源经过一系列中间交换机传输到目的地。

资源管理信元(RM cell)：散布在数据信元中，用来在主机和交换机之间**传递与拥塞相关的信息**。

发送方发出的RM信元，散布比率可调，默认值是**每32个数据信元中有一个RM信元**。

RM信元作用：用来提供直接网络反馈和经由接收方的网络反馈。当一个RM信元**到达目的地**时，将被“掉转方向”回送给发送方(可能做修改)；交换机也可以自己产生一个RM信元，并将该RM信元**直接发送给源**。

ATM ABR拥塞控制是基于速率的方法，即发送方明确地计算出它能发送的最大速率，并以此进行调整。

通知拥塞信息的三种机制：EFCI(显示转发拥塞指示)比特；CI(拥塞指示)和NI(无增长)比特；ER(显式速率)的设置。

3.7 TCP的拥塞控制

TCP是端到端的拥塞控制(网络层不参与)，由发送方感知网络拥塞，据此改变发送速率。无拥塞时增加发送速率，有拥塞时降低发送速率。

发送方通过判定感知网络拥塞，即**定时器超时**或者**收到3个重复ACK时**，降低发送速率。

发送方的拥塞窗口：随网络拥塞情况动态调整。TCP发送方通过发送窗口，控制已发送未确认的分组数不能超过接收窗口(表示接收方可用缓存大小，通过ACK获知)和拥塞窗口的最小值，实现流量控制和拥塞控制。

拥塞控制算法有：AIMD(加增倍减算法)，慢启动，超时事件后的保守机制等。